Natural Language Processing – Practical Session TP7

Mini Text Classification using Averaged Word Embeddings

Neural Word Embeddings

Dr. BENDIABDALLAH M.H
Université Belhadj BOUCHAIB – Ain Temouchent

General Idea

In this practical session, you will build a very small **text classification system** using **averaged word embeddings**. Instead of representing a sentence with a bag of words, you will:

1) Tokenise the sentence,
2) Convert each known word into its embedding vector,
3) Compute the average of all word vectors,
4) Use this averaged vector as the representation of the sentence,
5) Train a classifier to predict the sentence category.

This practical session applies the idea studied in the course This representation is simple , dense , low-dimensional , and useful for tasks such as **text classification** . However , it may lose some information about word order and negation.

Learning Objectives

By the end of this practical session, students should be able to:

- understand how to represent a sentence with averaged word embeddings;
- load pre-trained embeddings with Python;
- prepare a small labelled text dataset;
- transform texts into numeric vectors;
- train a simple machine learning classifier;
- evaluate classification results using accuracy and a classification report;
- analyse the strengths and limitations of averaged embeddings.

Practical Theme

We will classify short English sentences into one of the following categories:

- **Sports**
- **Technology**
- **Health**
- **Education**

Dr. BENDIABDALLAH M.H

1. Required Libraries

Make sure the following libraries are installed:

```
pip install gensim scikit-learn numpy pandas
```

2. Step 1 – Load the Dataset in Python

Task

Load a list of sentences and their corresponding labels.

```
csv_path = "train.csv"
df = pd.read_csv(csv_path, encoding="utf-8")


required_cols = {"text", "label"}
missing = required_cols - set(df.columns)
if missing:
    raise ValueError(f"Colonnes manquantes dans le CSV : {missing}")

texts = df["text"].astype(str).tolist()
labels = df["label"].astype(str).tolist()
```

3. Step 2 – Load a Pre-trained Word Embedding Model

In this TP, we use a pre-trained embedding model from `gensim.downloader`. A small model such as `glove-wiki-gigaword-50` is suitable for a classroom practical session.

```
import gensim.downloader as api

print("Loading embeddings...")
w2v = api.load("glove-wiki-gigaword-50")
print("Model loaded successfully!")
```

Important Note

The model dimension here is **50**. This means each known word is represented by a numeric vector of length 50.

4. Step 3 – Preprocess the Text

We will use a very simple preprocessing approach:

- convert text to lowercase;
- split text into words;
- remove punctuation in a simple way.

```
import re

def preprocess(text):
```

```
    text = text.lower()
    text = re.sub(r"[^a-zA-Z\s]", "", text)
    tokens = text.split()
    return tokens
```

5. Step 4 – Compute the Average Embedding of a Sentence

For each sentence:

1) preprocess the text,
2) keep only the words in the embedding vocabulary,
3) retrieve their vectors,
4) compute the mean vector.

If no word is found in the model vocabulary, we return a zero vector.

```
import numpy as np

def average_embedding(text, model):
    tokens = preprocess(text)
    vectors = []

    for token in tokens:
        if token in model:
            vectors.append(model[token])

    if len(vectors) == 0:
        return np.zeros(model.vector_size)

    return np.mean(vectors, axis=0)
```

6. Step 5 – Transform All Sentences into Vectors

```
X = np.array([average_embedding(text, w2v) for text in texts])
y = np.array(labels)

print("Shape of X:", X.shape)
print("First vector:", X[0])
```

Expected output:

- `X.shape` should be `(16, 50)`
- each sentence is now represented by one vector of size 50

Dr. BENDIAB AH M.H

7. Step 6 – Split the Dataset into Training and Test Sets

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.25,
    random_state=42,
    stratify=y
)

print("Training size:", len(X_train))
print("Test size:", len(X_test))
```

8. Step 7 – Train a Classifier

We will use **Logistic Regression**, which is simple and effective for small text classification tasks.

```
from sklearn.linear_model import LogisticRegression

clf = LogisticRegression(max_iter=1000)
clf.fit(X_train, y_train)

print("Classifier trained successfully!")
```

9. Step 8 – Evaluate the Classifier

```
from sklearn.metrics import accuracy_score, classification_report

y_pred = clf.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nClassification Report:\n")
print(classification_report(y_test, y_pred))
```

10. Step 9 – Test the Model on New Sentences

Now try the classifier on new sentences that were not in the training set.

```
new_df = pd.read_csv("test.csv", encoding="utf-8")
new_sentences = new_df["text"].astype(str).tolist()

new_vectors = np.array([average_embedding(s, w2v) for s in new_sentences])
predictions = clf.predict(new_vectors)

for sentence, pred in zip(new_sentences, predictions):
    print(f"Sentence: {sentence}")
    print(f"Predicted label: {pred}")
    print("-" * 50)
```

11. Step 10 – Display Prediction Probabilities

```python
probs = clf.predict_proba(new_vectors)

for i, sentence in enumerate(new_sentences):
    print(f"Sentence: {sentence}")
    for label, prob in zip(clf.classes_, probs[i]):
        print(f" {label}: {prob:.4f}")
    print("-" * 50)
```

Conclusion

In this practical session, you learned how to:

- use pre-trained word embeddings;
- represent a full sentence by averaging word vectors;
- train a simple classifier on these sentence vectors;
- evaluate a mini text classification system.

This TP is an important bridge between:

word representations $\longrightarrow$ **document representations** $\longrightarrow$ **real NLP applications**

Dr. BENDIABDALLAH M.H
Université Belhadj BOUCHAIB – Ain Temouchent

Dr.BENDIABDALLAH M.H